UNIVERSITY OF DHAKA

RedGrid: Dependency-Constrained UCB Exploration for Autonomous Penetration Testing

Submitted by

Exam Roll: 50028

Registration No: H-55

Exam Roll: 50040

Registration No: H-49

Submitted to the

Department of Computer Science and Engineering

University of Dhaka, Bangladesh

in partial fulfillment of the requirements for the degree of

Professional Masters in Information and Cyber Security (PMICS).

Declaration Form

University of Dhaka

We hereby declare that the work in this report titled "**RedGrid: Dependency-Constrained UCB Exploration for Autonomous Penetration Testing**" has been carried out by us under the supervision of Dr. Md. Abdur Razzaque. We further declare that where information has been derived from the literature, we have adequately cited and referenced the original sources. No part of this report was previously presented for another degree or diploma at this or any other institution.

Name of the Candidate
Nishan Paul

Name of the Candidate
Md Rakibur Rahman

Dr. Md. Abdur Razzaque
Professor and Chairman
Dept. of
Computer Science and Engg.
University of Dhaka

"To conquer fear, you must become fear."

Ra's al Ghul, Batman Begins

UNIVERSITY OF DHAKA

Abstract

Large Language Model (LLM) agents have repeatedly demonstrated the ability to exploit real-world software vulnerabilities, yet a systematic reading of the recent literature reveals two unresolved and independent failure modes. First, on the hardest realistic web benchmark (CVE-Bench), the best reported agent exploits only 13% of one-day and 10% of zero-day vulnerabilities, with *insufficient exploration* identified as the dominant cause of failure across every evaluated system. Second, on a stage-decomposed diagnostic benchmark (PentestEval), *Attack Decision-Making* – reasoning about which prerequisite weaknesses must be resolved before a candidate attack becomes feasible – is the single lowest-scoring stage of the pentesting pipeline. No system surveyed in the 29-paper corpus underlying this work addresses both failure modes at once: systems that solve exploration breadth dispatch flat, undifferentiated task lists with no explicit model of prerequisite structure, while systems that reason about dependencies do so only over small, pre-curated, expert-annotated weakness sets that cannot scale to open-ended discovery.

This report proposes **RedGrid**, a four-layer autonomous penetration-testing architecture whose central contribution is a dynamically constructed **Vulnerability Dependency Graph (VDG)** – a scored directed acyclic graph in which nodes represent candidate attack hypotheses and edges represent LLM-inferred prerequisite/enables relationships grown incrementally from live Specialist discovery. Node selection is governed by a modified Upper Confidence Bound (UCB1) formula restricted to a dependency-constrained eligible frontier, combined with path-level impact scoring so that a chain of moderate-value steps culminating in a high-impact outcome can outrank a single high-value but terminal node, and an ordinal evidence-confidence scale ($E_{\text{ord}} \in \{0, \ldots, 5\}$) that replaces uncalibrated raw LLM confidence in the scoring formula. The VDG sits inside a **Dual-Layer World Model** that

strictly separates confirmed environmental facts, written exclusively by six surface-specific Specialists (Reconnaissance, SQL Injection, Cross-Site Scripting, GraphQL, Authentication/Session, and Lateral Movement), from inferred attack hypotheses, written exclusively by a Team Manager – eliminating fact/hypothesis conflation and making discovery quality independently ablatable from planning quality. Supporting infrastructure includes a four-tier memory system (vulnerability-pattern, conditional-branching strategy, technical-action, and episodic-failure memory) with oracle-gated skill promotion and an explicit negative-transfer guard, a bounded Diagnosis-Adapt-Cap validation loop, and a context-reconstruction mechanism (FullCompact) that addresses the long-session context inflation documented in prior single-agent systems.

Following the scoping discipline established in the architecture, RedGrid claims exactly three contributions, each bounded by a precisely specified experimental test: **C1** – dependency-aware attack-graph exploration, validated by demonstrating that the unified VDG outperforms a stacked (filter-after-scoring) baseline on CVE-Bench and PentestEval; **C2** – cross-mission memory with verified skill promotion, validated by measurable improvement on a seen-technology benchmark subset; and **C3** – the first standardized cross-benchmark evaluation of a single autonomous VAPT architecture across web (CVE-Bench), GraphQL (PrediQL), and multi-host (Incalmo MHBench) attack surfaces under one shared orchestration layer with surface-specific Specialist pools and strictly separated per-surface reporting. This report presents the complete system architecture, the formalized VDG algorithm, the seven-tier benchmarking strategy, the required ablation design (with McNemar's test and 95% Wilson confidence intervals), a mandatory pre-evaluation gate on VDG edge-inference precision, and an honest accounting of the threats to validity – including the expected gap between sandboxed and real-world pass rates – that will govern the empirical evaluation to follow.

Acknowledgements

We would like to thank ...

Contents

List of Figures

List of Tables

List of Algorithms

Acronym	**W**hat (it) **S**tands **F**or
LAH	**L**ist **A**bbreviations **H**ere

Constant Name	Symbol	=	Constant Value (with units)
Speed of Light	c	=	$2.997\ 924\ 58 \times 10^8\ \mathrm{ms}^{-\mathrm{S}}$ (exact)

symbol	name	unit
a	distance	m
P	power	W (Js^{-1})
ω	angular frequency	rads^{-1}

For/Dedicated to/To my...

Chapter 1

Introduction

1.1 Background and Motivation

Penetration testing – the authorized, systematic attempt to discover and exploit weaknesses in a target system before a real adversary does – has traditionally resisted automation because it demands the kind of open-ended, context-sensitive reasoning that scripted scanners cannot perform. Over the past two years, agentic Large Language Models (LLMs) have repeatedly closed parts of this gap. Fang et al. [3] showed that a single LLM agent equipped with a browser and a small set of tools could autonomously perform multi-step SQL-injection attacks against live websites without being told which vulnerability to look for. Fang et al. [4] extended this to real, critical-severity CVEs, showing that GPT-4 could exploit 87% of a 15-vulnerability benchmark when given the CVE description, though this collapsed to 7% once the description was withheld – the first clear evidence that *exploiting* a known vulnerability and *discovering* one are almost entirely different capabilities. Zhu et al. [5] pushed further into genuinely zero-day territory with a hierarchical team of planning and task-specific agents (HPTSA), and a wide subsequent literature – surveyed systematically in Chapter 2 – has proposed increasingly elaborate multi-agent, memory-augmented, and planning-augmented architectures for autonomous vulnerability assessment and penetration testing (VAPT).

A systematic reading of this literature – a corpus of 29 papers spanning single-agent exploitation studies, hierarchical multi-agent frameworks, classical-planning hybrids, cross-mission memory systems, and seven independent benchmark suites – together with four independently drafted agent-architecture proposals and expert adjudication between them, is the foundation on which this report is built. That reading converges on one finding echoed independently by at least six of the surveyed systems (AWE, AutoPT, VulnBot, PentestAgent, D-CIPHER, and Incalmo): **architecture, not model scale, is the dominant variable in autonomous exploitation performance.** A well-structured pipeline running a comparatively inexpensive model consistently outperforms an unstructured ReAct-style loop running a frontier model. Yet the same reading surfaces two independent, unresolved failure modes that no single surveyed system addresses simultaneously, and it is the combination of these two gaps – not either one alone – that motivates the system proposed in this report.

1.2 Problem Statement

Failure Mode 1 – Insufficient exploration. On CVE-Bench [1], a benchmark of 40 critical (CVSS $\geq$ 9.0) real-world web-application CVEs, the strongest reported agent exploits only 13% of one-day and 10% of zero-day vulnerabilities. CVE-Bench's own failure-mode breakdown identifies *insufficient exploration* as the dominant cause of failure across every agent and evaluation setting, with exploration-failure rates ranging from 37.5% to 80.0% (T-Agent: 80.0% zero-day / 55.0% one-day; AutoGPT: 72.5% / 45.0%; Cy-Agent: 67.5% / 37.5%), as illustrated in Figure 1.1. Crucially, this is not a reasoning-quality problem – it is a breadth-of-search problem: agents commit early to a narrow set of candidate attack paths and never return to explore the rest of the surface.

Failure Mode 2 – The dependency-reasoning gap. PentestEval [2] decomposes the pentesting pipeline into six stages – Information Collection (IC), Weakness Gathering (WG), Weakness Filtering (WF), Attack Decision-Making (ADM),

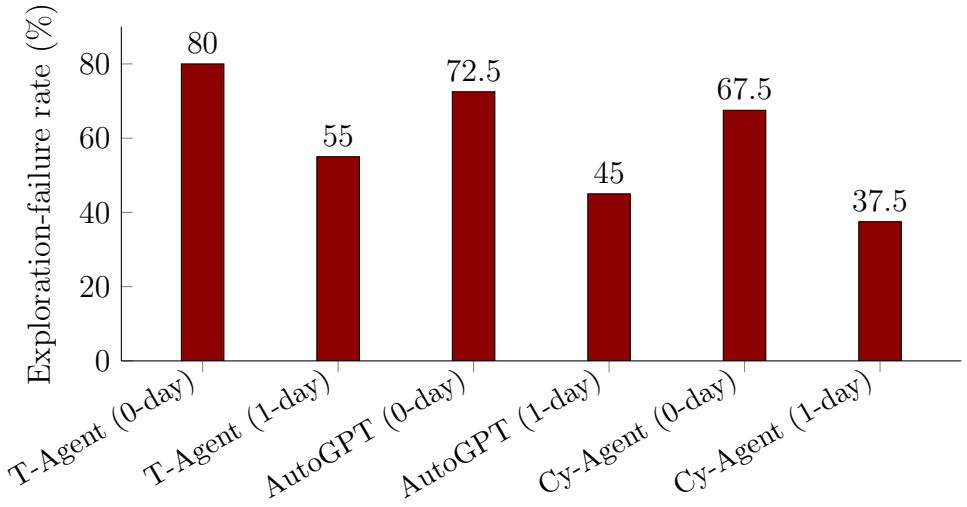

FIGURE 1.1: Exploration-failure rate as the dominant CVE-Bench failure mode, by agent and mode (data from CVE-Bench's Table 5 failure-mode breakdown [1]).

Exploit Generation (EG), and Exploit Revision (ER) – and finds the opposite bottleneck at the *planning* level: ADM, which requires reasoning about prerequisite dependencies between candidate weaknesses, is the single lowest-scoring stage (Spearman correlation 0.25 against expert judgement). PentestEval's ground-truth-injection ablation quantifies exactly how much is available at each stage, summarized in Table 1.1.

TABLE 1.1: PentestEval ground-truth-injection ablation: marginal gain from replacing each stage's output with expert ground truth, applied cumulatively on top of the SMP baseline [2].

Configuration	**End-to-end success**	**Marginal gain**
SMP (baseline)	0.31	–
+ GT Weakness Gathering (WG)	0.50	+0.19
+ GT Weakness Filtering (WF)	0.53	+0.03
+ GT Attack Decision-Making (ADM)	0.67	**+0.14**

ADM delivers the largest single-stage marginal increment (+0.14) of the three tested – measured, notably, on top of an already ground-truthed WG+WF pipeline, not in isolation. Any dynamically-grown dependency structure is necessarily a weaker approximation than ground-truth ADM, so its performance ceiling must be reported honestly as below 0.67 and bounded by the quality of its LLM-inferred edges.

The gap no surveyed system closes. Systems that solve exploration breadth (T-Agent, HPTSA [5], MAPTA [6]) dispatch flat, undifferentiated task lists with no explicit prerequisite or dependency model. Systems that solve dependency reasoning (PentestEval's SMP baseline, CHECKMATE-style classical-planning approaches [7]) are evaluated on curated scenarios with pre-enumerated weakness sets and do not scale to open-ended, wide-surface exploration. No system in the surveyed corpus combines UCB-guided exploration over a dependency-constrained frontier, prerequisite/enables edges grown dynamically from live Specialist discovery, cross-session verified skill accumulation, and evaluation across three independently-benchmarked attack-surface families in one architecture. Closing this compound gap is the problem this report addresses.

1.3 Objectives

This report defines, at implementation level, an autonomous VAPT architecture – **RedGrid** – and its accompanying evaluation methodology. The concrete objectives are:

1. To design a **Vulnerability Dependency Graph (VDG)** that unifies dependency-aware exploration with UCB-guided node selection, path-level impact scoring, and failure propagation, formalized to pseudocode level (Chapter 3).

2. To design a **Dual-Layer World Model** that strictly separates confirmed environmental facts from inferred attack hypotheses, with enforced write-ownership, so that discovery quality and planning quality can be ablated independently.

3. To design a **four-layer orchestration architecture** (Orchestrator, Team Manager, Specialists, Execution/Validation) instantiating the structural pattern that every high-performing surveyed system independently converges on.

4. To design a **cross-mission memory system** with security-specific conditional branching (e.g., WAF-adaptive payload switching) and oracle-gated

skill promotion, distinguishing genuinely reusable strategies from single-target idiosyncrasies.

5. To define a rigorous, fully pre-existing-benchmark-based **evaluation plan** – spanning web, GraphQL, and multi-host attack surfaces – with statistical methodology (McNemar's test, 95% Wilson confidence intervals, compute normalization) sufficient for systems-and-empirical-evaluation publication.

6. To specify the **ablation studies** required to support each contribution claim, so that no claim in the eventual evaluation is made without a corresponding controlled experiment.

1.4 Scope of the Project

RedGrid applies one governing rule throughout: it targets *only* attack surfaces for which a dedicated, reusable, oracle-backed benchmark already exists in the surveyed literature. No new benchmark is constructed for this project. Table 1.2 summarizes the four in-scope attack-surface families and the benchmarks that bound every claim made about them.

TABLE 1.2: In-scope attack surfaces and their governing benchmarks.

Attack surface	**Benchmark(s)**	**Representative coverage**
Web application (HTTP/HTML)	Fang et al. 15-vuln sandbox [4]; HPTSA 14-CVE zero-day suite [5]; CVE-Bench (40 CVEs) [1]; MAPTA/XBOW (104) [6]; HackWorld (36) [8]; PentestEval (346 tasks) [2]; Cybench [9]; PentestGPT/HTB machine sets [10]	SQLi, XSS, CSRF, SSRF, SSTI, LFI, file-upload RCE, IDOR, auth bypass, framework RCEs, JWT forgery
GraphQL APIs	PrediQL 6-API suite [11]	Schema abuse, dependency-chain injection, batched-auth bypass, IDOR, nested-query DoS
Multi-host / Active Directory	Incalmo MHBench (40 environments) [12]	Lateral movement, credential reuse, privilege escalation
Production system corpus (hard tier)	BountyBench (25 real systems) [13]	Economic/adversarial validation layer over the web surface

Explicitly out of scope. General REST API exploitation is not evaluated and no REST-specific pass rate is ever reported: RESTler's [14] evaluation targets are one-off real-world case studies rather than a standardized, reusable benchmark, so there is no "RESTBench" equivalent in the surveyed corpus. RESTler's dependency-inference and response-feedback-pruning techniques are nonetheless methodologically valuable and are reused *internally* by the GraphQL Specialist and by the Team Manager's edge-inference logic (Section 3.3). Binary exploitation, physical- and network-layer attacks, and social engineering are likewise out of scope for the same reason: no dedicated, oracle-backed benchmark for them exists in the surveyed literature.

1.5 Expected Contributions

Consistent with the design principle that a paper claiming many contributions invites the conclusion that it is a "bundle of incremental integrations," this report makes exactly three primary scientific contributions, each bounded by a precisely specified experimental test (formalized in Chapter 5):

- **C1 – Dependency-Aware Attack Graph Exploration (Primary).** A dynamically constructed VDG combining UCB-guided selection over a dependency-constrained frontier, path-level impact scoring, and ordinal evidence back-propagation, targeting zero-day pass@1 $\geq 25\%$ on CVE-Bench and an ADM score ≥ 0.50 on PentestEval, contingent on a mandatory pre-evaluation edge-inference precision gate.
- **C2 – Cross-Mission Memory with Verified Skill Promotion (Supporting).** A three-tier memory architecture with security-specific conditional-branching strategies and oracle-gated skill promotion, targeting measurable improvement on seen-technology targets relative to a no-memory baseline.
- **C3 – Comprehensive Cross-Benchmark Evaluation with Standardized Oracles (Methodological).** The first evaluation of a single autonomous

VAPT architecture across web, GraphQL, and multi-host surfaces under one shared orchestration layer, with surface-specific Specialist pools and strictly separated per-surface reporting.

Several further mechanisms – declarative task dispatch, the Structured Handoff Bridge, the Diagnosis-Adapt-Cap validation loop, the Usage Tracker, and the Engagement Trajectory Log – are retained as supporting infrastructure that the evaluation depends on, but are deliberately *not* claimed as primary novelty contributions; the rationale for each demotion is given in Section 1.5.

1.6 Challenges

Nine threats to validity are tracked explicitly throughout this report and are treated as first-class design constraints rather than post-hoc caveats (full detail in Section 1.6):

1. **VDG edge-inference accuracy** is LLM-inferred rather than ground-truth-annotated, and is the single highest-risk item in the entire architecture; a mandatory pilot study against PentestEval's ground-truth dependency annotations must show $\geq 50\%$ precision before the main evaluation may proceed.
2. **Sandbox-to-real-world gap.** Fang et al. [3] found 1 exploitable XSS in 50 real-world candidate sites (2%) versus 73.3% in a matched sandbox; RedGrid must report both regimes and must not extrapolate from one to the other.
3. **GraphQL evaluation is statistically narrow** – PrediQL's 6-API suite is real and standardized but far smaller than CVE-Bench or XBOW, so generalization claims involving GraphQL must state this asymmetry plainly.
4. **Cost-per-exploit is backbone-price-sensitive** and must be reported with an explicit model, version, and date rather than as an absolute claim.

5. **Negative transfer in cross-mission memory** – a strategy validated against one framework version may be actively harmful against another; this risk is unaddressed in all 29 surveyed papers because none implement cross-mission, strategy-level memory generalization for VAPT.

6. **UCB hyperparameter sensitivity** across seven tunable parameters, requiring an explicit search-range report and a $\pm 10\%$ perturbation sensitivity analysis.

7. **Statistical power on small benchmarks** – PrediQL's 6 APIs at 5 runs yield only 30 data points, likely underpowering McNemar's test for small effect sizes.

8. **Edge-inference scalability** – unbatched pairwise edge inference would require $O(2M)$ frontier-model calls per new node, plausibly exceeding the entire per-CVE compute budget; batching is required and its own quality trade-offs must be measured.

9. **Honest framing of "one architecture, three surfaces"** – different Specialist pools activate per attack surface, so C3 is reported as a shared orchestrator with surface-specific modules, not as one literally unmodified architecture.

1.7 Organization

The remainder of this report is organized as follows. Chapter 2 surveys the 29-paper corpus underlying this work, organizing it by architectural pattern, and derives the fifteen-item prior-work gap table that motivates RedGrid's design. Chapter 3 presents the complete RedGrid architecture: the four-layer orchestration hierarchy, the Dual-Layer World Model, the formalized VDG algorithm (node schema, UCB formula, edge-construction and update rules, path scoring, and failure propagation), the per-Specialist methodology for each of the four in-scope attack surfaces, and the memory and state services. Chapter 4 lays out the execution plan for building and

validating this architecture – work breakdown, timeline, milestones, resourcing, and risk assessment. Chapter 5 specifies the evaluation plan: the seven-tier benchmark suite, the statistical methodology, and the eight required ablation studies. Chapter 6 summarizes the report, lists the expected deliverables, and outlines directions for future work beyond the scope defined here.

Chapter 2

Literature Review and Related Work

2.1 Domain Overview

2.2 Existing Systems

2.3 Comparative Analysis

2.4 Research Gap

2.5 Summary

Chapter 3

Proposed Methodologies

3.1 System Overview

3.2 Requirement Analysis

3.3 Proposed Architecture

3.4 System Workflow

3.5 Tools and Technologies

Chapter 4

Execution Plan

4.1 Work Breakdown Structure

4.2 Project Timeline

4.3 Milestones

4.4 Resource Requirements

4.5 Risk Assessment

Chapter 5

Experimental Results

5.1 Evaluation Plan

Chapter 6

Conclusions

6.1 Summary

6.2 Expected Deliverables

6.3 Future Works

Bibliography

[1] Yuxuan Zhu, Antony Kellermann, Dylan Bowman, Philip Li, Akul Gupta, Adarsh Danda, Richard Fang, Conner Jensen, Eric Ihli, Jason Benn, Jet Geronimo, Avi Dhir, Sudhit Rao, Kaicheng Yu, Twm Stone, and Daniel Kang. CVE-Bench: A benchmark for AI agents' ability to exploit real-world web application vulnerabilities. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, 2025.

[2] Ruozhao Yang, Mingfei Cheng, Gelei Deng, Tianwei Zhang, Junjie Wang, and Xiaofei Xie. PentestEval: Benchmarking LLM-based penetration testing with modular and stage-level design. Preprint, 2025.

[3] Richard Fang, Rohan Bindu, Akul Gupta, Qiusi Zhan, and Daniel Kang. LLM agents can autonomously hack websites. arXiv preprint arXiv:2402.06664, 2024.

[4] Richard Fang, Rohan Bindu, Akul Gupta, and Daniel Kang. LLM agents can autonomously exploit one-day vulnerabilities. arXiv preprint arXiv:2404.08144, 2024. v2, cs.CR.

[5] Yuxuan Zhu, Antony Kellermann, Akul Gupta, Philip Li, Richard Fang, Rohan Bindu, and Daniel Kang. Teams of LLM agents can exploit zero-day vulnerabilities. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2026. arXiv:2406.01637.

[6] Isaac David and Arthur Gervais. Multi-agent penetration testing AI for the web. arXiv preprint arXiv:2508.20816, 2025.

[7] Lingzhi Wang, Xinyi Shi, Ziyu Li, Yi Jiang, Shiyu Tan, Yuhao Jiang, Junjie Cheng, Wenyuan Chen, Xiangmin Shen, Zhenyuan Li, and Yan Chen. Automated penetration testing with LLM agents and classical planning. arXiv preprint arXiv:2512.11143, 2025.

[8] Xiaoxue Ren, Penghao Jiang, Kaixin Li, Zhiyong Huang, Xiaoning Du, Jiaojiao Jiang, Zhenchang Xing, Jiamou Sun, and Terry Yue Zhuo. HackWorld: Evaluating computer-use agents on exploiting web application vulnerabilities. arXiv preprint arXiv:2510.12200, 2025.

[9] Andy K. Zhang, Neil Perry, Riya Dulepet, Joey Ji, Celeste Menders, Justin W. Lin, Eliot Jones, Gashon Hussein, Samantha Liu, Donovan Jasper, Pura Peetathawatchai, Ari Glenn, Vikram Sivashankar, Daniel Zamoshchin, Leo Glikbarg, Derek Askaryar, Mike Yang, Teddy Zhang, Rishi Alluri, Nathan Tran, Rinnara Sangpisit, Polycarpos Yiorkadjis, Kenny Osele, Gautham Raghupathi, Dan Boneh, Daniel E. Ho, and Percy Liang. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[10] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*, pages 847–864, Philadelphia, PA, 2024. USENIX Association.

[11] Shaolun Liu, Sina Marefat, Omar Tsai, Yu Chen, Zecheng Deng, Jia Wang, and Mohammad A. Tayebi. PrediQL: Automated testing of GraphQL APIs with LLMs. arXiv preprint arXiv:2510.10407, 2025.

[12] Brian Singer, Keane Lucas, Lakshmi Adiga, Meghna Jain, Lujo Bauer, and Vyas Sekar. Incalmo: An autonomous LLM-assisted system for red teaming multi-host networks. arXiv preprint arXiv:2501.16466, 2025.

[13] Andy K. Zhang, Joey Ji, Celeste Menders, Riya Dulepet, Thomas Qin, Ron Y. Wang, Junrong Wu, Kyleen Liao, Jiliang Li, Jinghan Hu, Sara Hong, Nardos

Demilew, Shivatmica Murgai, Jason Tran, Nishka Kacheria, Ethan Ho, Denis Liu, Lauren McLane, Olivia Bruvik, Dai-Rong Han, Seungwoo Kim, Akhil Vyas, Cuiyuanxiu Chen, Ryan Li, Weiran Xu, Jonathan Z. Ye, Prerit Choudhary, Siddharth M. Bhatia, Vikram Sivashankar, Yuxuan Bao, Dawn Song, Dan Boneh, Daniel E. Ho, and Percy Liang. BountyBench: Dollar impact of AI agent attackers and defenders on real-world cybersecurity systems. In *Advances in Neural Information Processing Systems (NeurIPS) 39, Track on Datasets and Benchmarks*, 2025. arXiv:2505.15216.

[14] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. RESTler: Stateful REST API fuzzing. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 748–758. IEEE, 2019.

Appendix A: RedGrid Reference Material

This appendix provides full specifications for the RedGrid system that would exceed the length constraints of the main chapters, including the complete VDG node schema, benchmark suite details, UCB hyperparameter search configuration, and ablation condition pseudocode.

.1 Full VDG Node Schema

Table 1 provides the complete field listing for a VDG node as defined in Chapter 3. Every field is populated by the Team Manager via `VDG_AddNode`; no Specialist writes directly to the VDG.

Table 1: Complete VDG Node Schema

Field	Type	Description
`weakness_id`	`str`	Unique identifier (e.g. `sqli-001`)
`vuln_class`	`enum`	`SQLi | XSS | CSRF | SSTI | LFI | AuthBypass | IDOR | RCE | GraphQL_Injection | LateralMove`
`attack_intent`	`str`	Natural language description of the goal

Field	Type	Description
`prerequisites`	`List[str]`	`weakness_id`s that must reach `EXPLOITED` before this node is eligible
`enables`	`List[str]`	`weakness_id`s that become `ELIGIBLE` once this node is `EXPLOITED`
`status`	`enum`	`ELIGIBLE` | `IN_PROGRESS` | `EXPLOITED` | `INFEASIBLE` | `BLOCKED`
`w`	`float`	Cumulative UCB reward (sum of outcome scores)
`n`	`int`	Times this node has been selected (initialised to 1 to avoid division by zero)
`phi`	`float`	Promise score $\phi \in [0, 1]$: LLM-assessed exploitability likelihood
`delta`	`float`	Task difficulty index $\delta \in [0, 1]$; 1 = maximally hard
`E_ord`	`int`	Ordinal evidence score $E_{\text{ord}} \in \{0, 1, 2, 3, 4, 5\}$ (see Table 2)
`epss_prior`	`float`	EPSS score $\in [0, 1]$ from NVD/FIRST API; default 0.05 if no CVE match
`context_load`	`float`	Estimated Specialist context cost $C \in [0, 1]$
`estimated_cost`	`float`	Estimated token cost for this node's exploitation
`retry_count`	`int`	Number of Specialist invocations on this node
`max_retries`	`int`	Default 3; on exhaustion $\rightarrow$ `INFEASIBLE`
`path_score`	`float`	Score of best feasible path this node participates in

Field	Type	Description
`source_el_nodes`	`List[str]`	EL node IDs that seeded this VDG node
`last_updated`	`timestamp`	Timestamp of last field mutation

.2 Ordinal Evidence Score (E_{ord}) Reference

Table 2 is the full rubric used by the Evaluation Agent to assign E_{ord} after every Specialist task. The value is parsed deterministically from constrained JSON output — not inferred from free-form text.

TABLE 2: Ordinal Evidence Score (E_{ord}) Rubric

E_{ord}	Label	Criteria
0	Unseen	Node not yet acted on
1	Tool ran / nothing	Tool executed; target responded normally; no anomaly detected
2	Weak signal	Anomalous response (different status code, timing difference) but ambiguous
3	Clear indication	Behavioural evidence consistent with vulnerability (e.g. SQL error message, reflected input) but not yet controlled
4	Confirmed behaviour	Controlled behaviour demonstrated (e.g. parameterised timing differential, payload executing in non-live context)
5	Oracle-confirmed	Per-surface oracle confirms successful exploitation

.3 Benchmark Suite Summary

Table 3 summarises all eight benchmark tiers used in REDGRID's evaluation. No benchmark was constructed for this work; every tier references a published, reusable target set.

Table 3: Full RedGrid Benchmark Suite

Tier	Benchmark	Surface	Size	Role
0	Fang et al. sandbox	Web	15 CVEs	CI regression flo
0b	HPTSA 14-CVE suite	Web (0-day)	14 CVEs	Zero-day mode v tion
1	PentestEval	Web	12 / 346	Stage-level diag UCB tuning
2	CVE-Bench	Web	40 CVEs	**Primary m** (pass@1, pass@5
2b	MAPTA / HackWorld / Cybench	Web	∼180	Cross-benchmar eralisation
3	PrediQL	GraphQL	6 APIs	Schema-coverage baselines
4	MHBench	Multi-host	40 envs	Host-compromis credential-theft
5	BountyBench	Web (prod.)	25 systems	Dollar-value cost-per-exploit
6	PentestGPT set + HTB S8	Web	18 machines	Live-competitio dation

.4 UCB Hyperparameter Search Grid

The UCB formula (Chapter 3) has seven tunable parameters. Table 4 documents the search ranges and defaults. Grid search is performed on Tier 1 (PentestEval) only, holding out the CVE-Bench split.

Results must be reported with sensitivity analysis: ±10% perturbation from each optimal value while holding others fixed.

Table 4: UCB Hyperparameter Search Grid

Parameter	Meaning	Search Range	Default
C_{expl}	Exploration constant	$\{0.5, 1.0, 1.414, 2.0\}$	1.414
α	Weight for promise ϕ	$[0.1, 0.5]$ step 0.1	0.3
β	Weight for difficulty $(1 - \delta)$	$[0.1, 0.5]$ step 0.1	0.2
γ	Weight for $E_{\text{ord}}/5$	$[0.1, 0.5]$ step 0.1	0.4
κ	Context-load penalty	$[0.05, 0.2]$ step 0.05	0.1
λ	EPSS prior weight	$[0.05, 0.3]$ step 0.05	0.15
μ	Cost-awareness penalty	$[0.05, 0.2]$ step 0.05	0.1

.5 Ablation Condition A1 — Precise Pseudocode

This section provides the exact algorithmic distinction between Ablation A1 conditions (b) and (c), which is critical for interpreting whether the unified VDG adds value over a stacked approach.

Condition (b) — UCB with dependency-constrained eligible set (pre-filter):

```
eligible = [v for v in VDG.nodes
            if v.status == ELIGIBLE
            and all_prerequisites_satisfied(v)]
scores   = {v: UCB_score(v, N=len(eligible))
            for v in eligible}
selected = max(eligible, key=lambda v: scores[v])
```

Condition (c) — Stacked: UCB over all nodes, post-filter by dependency (the key discriminant):

```
all_nodes = VDG.all_unattempted_nodes()
scores    = {v: UCB_score(v, N=len(all_nodes))
             for v in all_nodes}
ranked    = sorted(all_nodes,
                   key=lambda v: scores[v],
```

```
                    reverse=True)
filtered  = [v for v in ranked
             if all_prerequisites_satisfied(v)]
selected  = filtered[0]
```

If condition (d) (full VDG with path scoring) outperforms condition (c), the unified graph structure has value beyond dependency filtering alone. If (d) $\approx$ (c), the contribution downgrades to dependency-aware UCB filtering — still a contribution, but a narrower claim. This gate must be reported honestly.

.6 Prior Work Gap Summary

Table 5 consolidates the gap analysis from Chapter 2 into a compact reference table.

Table 5: Prior Work Gaps and RedGrid Fixes

Gap	**Papers exhibiting**	**RedGrid fix**
Flat dispatch, no prerequisite modelling	HPTSA, MAPTA, T-Agent	VDG dependency-constrained frontier
Dependency planning on pre-curated sets only	PentestEval SMP, CHECKMATE	Dynamic `VDG_AddNode` from Specialist discovery
Node-level scoring misses multi-step chains	EGATS	Path scoring: product of node scores $\times$ impact / cost
No failure propagation	All 29 surveyed papers	`BLOCKED` propagation + frontier recomputation
Exploration failure unaddressed architecturally	CVE-Bench (diagnostic)	Full-surface Recon + dependency-constrained frontier
Session/multi-turn state loss	Fang et al., PentestGPT	Auth/Session Specialist + Session Persistence Service

Gap	**Papers exhibiting**	**RedGrid fix**
Context pollution from rolling history	PentestGPT, D-CIPHER, VulnBot	Fresh context per Specialist invocation
Long-session context inflation	PentestGPT Finding 4	FullCompact: EL+AL-based reconstruction at 85%
Raw LLM confidence overconfident in UCB	EGATS	E_{ord} ordinal scale (0–5) replaces raw confidence
Single-attempt validation: false positives	All single-agent systems	Diagnosis-Adapt-Cap loop (up to 3 retries)
Per-mission failure repetition	All fresh-context designs	Episodic Failure Memory (4th FAISS tier)
No cross-mission skill promotion	AWE, PrediQL, VulnBot	3-tier memory + oracle-gated skill promotion
Single benchmark surface evaluated per paper	All 29 papers	Web + GraphQL + multi-host under one orchestrator